Assignment No. 1

Assignment Title: Create a Java program using JDBC (Java Database Connectivity) to connect to a database, retrieve data from a table, and display the results.

Aim: Let's consider a scenario where we have a MySQL database named "example_db" with a table named "employees". The "employees" table has three columns: "id", "name", and "salary". We want to retrieve all records from this table and display them using JDBC in a Java program.

Pre-Requisites:

- Basics of Java programming.
- Understanding of SQL queries (SELECT, CREATE, INSERT).
- Familiarity with database concepts and table structures.

Objective:

1. Establish a connection with a MySQL database.
2. Execute SQL queries to retrieve data from the "employees" table.
3. Display the retrieved data in a readable format in the console.

Outcomes:

1. Understand how to use JDBC to connect to a database.
2. Learn how to execute SQL queries from Java code.
3. Gain experience handling database operations like connection, statement creation, and result processing.
4. Be able to manage resources efficiently using try-catch-finally blocks.
5. Appreciate the integration of Java applications with relational databases like MySQL.

Theory:

What is JDBC?

JDBC (Java Database Connectivity) is an API provided by Java to interact with relational databases. It acts as a bridge between Java applications and databases by providing a standard interface for database operations.

Key Components of JDBC:

1. DriverManager: Manages database drivers and establishes connections to databases.
2. Connection: Represents an active connection to the database.
3. Statement/PreparedStatement: Used to execute SQL queries.
4. ResultSet: Stores the result of a query.
5. SQLException: Handles errors or issues during database operations.

1. DriverManager

- What It Is:
  The DriverManager class is responsible for managing a list of database drivers. It provides methods to register drivers and establish connections to a database.
- Responsibilities:
  - Manages multiple database drivers for various databases (e.g., MySQL, Oracle, PostgreSQL).
  - Matches the requested connection URL with an appropriate driver.
- How It Works:
  - When you call DriverManager.getConnection(), it checks all registered drivers to find one that matches the database URL.
  - Once a suitable driver is found, a connection to the database is established.
- Code Example:

```
Connection connection
= DriverManager.getConnection("jdbc:mysql://localhost:3306/example_db", "username",
"password");
```

2. Connection

- What It Is:
  The Connection interface represents an active connection between a Java application and the database. It is the backbone of all database operations.
- Responsibilities:
  - Enables communication with the database.
  - Provides methods to create Statement and PreparedStatement objects.
  - Manages transactions (e.g., commit, rollback).

- Allows for setting connection properties (e.g., auto-commit mode).

- Code Example:

```
Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/example_db", "root", "password");

connection.setAutoCommit(false); // Example of managing transactions.
```

Important Note: Always close the connection after use to release resources:

```
connection.close();
```

3. Statement and PreparedStatement

- Statement:
  - What It Is:
    The Statement interface is used to execute static SQL queries against the database.
  - Responsibilities:
    - Sends SQL commands to the database.
    - Retrieves results for queries like SELECT.
  - Limitations:
    - Vulnerable to SQL injection if user inputs are not validated.
  - Code Example:

```
Statement statement = connection.createStatement();

ResultSet resultSet = statement.executeQuery("SELECT * FROM employees");
```

PreparedStatement:

- What It Is:
  A subclass of Statement that allows parameterized SQL queries, improving security and performance.
- Advantages:
  - Prevents SQL injection.
  - Optimized by the database (reusable precompiled queries).
- Code Example

```
PreparedStatement preparedStatement = connection.prepareStatement("SELECT * FROM employees WHERE id = ?");

preparedStatement.setInt(1, 101);

ResultSet resultSet = preparedStatement.executeQuery();
```

4. ResultSet

- What It Is:
  The ResultSet interface represents the data retrieved from the database. It is essentially a table of rows and columns.
- Responsibilities:
  - Iterates through the results of a query.
  - Provides methods to retrieve data by column index or name.
  - Supports moving the cursor through rows (e.g., next(), previous()).
- Code Example:

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM employees");

while (resultSet.next()) {
    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    double salary = resultSet.getDouble("salary");
    System.out.println("ID: " + id + ", Name: " + name + ", Salary: " + salary);
}
```

Important Methods:

- next(): Moves to the next row.
- getString(), getInt(), etc.: Retrieves data from a specific column.
- close(): Frees up resources associated with the ResultSet.

5. SQLException

- What It Is:
  SQLException is an exception class in Java that handles database-related errors during runtime.
- Responsibilities:

- Indicates issues like connection failures, incorrect SQL syntax, or access violations.
- Provides detailed information about the error through methods like getMessage(), getErrorCode(), and getSQLState().

- How to Handle: Always use try-catch blocks to handle SQLException gracefully.

```
try {
    Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/example_db", "root", "password");
    Statement statement = connection.createStatement();
    ResultSet resultSet = statement.executeQuery("SELECT * FROM employees");
} catch (SQLException e) {
    System.out.println("Error: " + e.getMessage());
    e.printStackTrace();
}
```

Steps to Use JDBC:

1. Load the JDBC driver.
2. Establish a connection to the database.
3. Create a statement object.
4. Execute SQL queries and retrieve results.
5. Process the ResultSet data.
6. Close the connection to release resources.

Conclusion:

This assignment illustrates the practical implementation of Java Database Connectivity (JDBC). By retrieving and displaying data from the "employees" table, you've learned how to interact with databases through Java.

Frequently Asked Questions:

1. What are the main components of a JDBC program, and how do they work together?
2. What is the role of the JDBC URL, and how should it be formatted?

3. What is the role of DriverManager in JDBC, and how does it interact with the database driver? Give syntax.
4. What are some real-world use cases of JDBC in software applications? Explain anyone in detail.
5. What is the difference between executeQuery(), executeUpdate(), and execute() in JDBC? When should each method be used?

Assignment No. 2

Assignment Title

Design and Implementation of a Java-Based GUI Login System with Database Connectivity

Aim

Create a Java-based GUI application that includes a user login page with full database connectivity. The application should allow users to enter their username and password, validate the credentials against a connected database, and display appropriate success or failure messages.

Pre-Requisites

- Basic understanding of Java programming
- Knowledge of Java Swing
- Understanding of JDBC
- Basic SQL knowledge
- Familiarity with relational databases
- Event handling in Java

Objective

- To create a user-friendly login interface
- To establish secure database connectivity
- To validate user credentials
- To display success or error messages
- To understand real-world authentication mechanisms

Outcomes

- Ability to design GUI applications using Java Swing
- Implement database-driven authentication
- Use JDBC APIs efficiently
- Handle exceptions and user events
- Develop secure Java desktop applications

Theory

Java Swing is used to create platform-independent GUI applications. A login system validates users by comparing entered credentials with stored database records. JDBC enables Java applications to connect with databases and execute SQL queries securely using PreparedStatement. The authentication process ensures authorized access and prevents misuse.

7. Conclusion

This assignment demonstrates the development of a Java-based GUI login system integrated with a database. It provides hands-on experience in GUI design, database connectivity, and authentication mechanisms, forming a foundation for real-world Java application development.

Frequently Asked Questions:

1. What is Java Swing?

2. What are the key features of Java Swing?

3. How is Swing different from AWT?

4. What are some commonly used Swing components?

5. What is the MVC architecture in Swing?

Assignment No. 3

Assignment Title: Create a Java GUI application to calculate the simple interest based on user input for principal amount, rate of interest, and time period (JAVA GUI).

Aim: Create a Java GUI application to calculate the simple interest based on user input for principal amount, rate of interest, and time period.

Pre-Requisites:

1. Basic Java Programming
2. Mathematical Concepts
3. Java Swing Basics
4. Event Handling in Java
5. IDE Setup
6. Error Handling

Objective:

1. To design and develop a Java GUI application using the Swing framework.
2. To enable users to input values for Principal Amount, Rate of Interest, and Time Period and compute the Simple Interest using the formula: SI=P×R×T100SI = \frac{P \times R \times T}{100}SI=100P×R×T
3. To demonstrate the practical use of Swing components such as JFrame, JLabel, JTextField, and JButton to build an interactive and user-friendly interface.
4. To implement event handling through the ActionListener interface for button click actions.
5. To showcase the application of error handling in validating user inputs and providing appropriate feedback using JOptionPane.
6. To enhance problem-solving and programming skills by integrating mathematical computations into GUI applications.

Outcomes:

1. Design and develop a GUI application using Java Swing to solve real-world problems, such as calculating financial metrics.
2. Apply Swing components like JFrame, JLabel, JTextField, and JButton effectively in creating interactive user interfaces.
3. Implement event handling using the ActionListener interface to respond to user actions such as button clicks.

4. Validate user inputs to ensure error-free calculations and provide feedback for invalid entries.
5. Perform mathematical computations programmatically, such as calculating simple interest.
6. Enhance debugging and error-handling skills in Java to build robust applications.
7. Gain practical experience in Java GUI programming, improving confidence in creating standalone desktop applications.

Theory:

The Simple Interest Calculator is a graphical user interface (GUI) application developed using Java Swing. It demonstrates how to create an interactive application to perform financial calculations, focusing on both user experience and functionality.

1. Simple Interest Formula

The formula to calculate simple interest is:

SI=P×R×T100SI = \frac{P \times R \times T}{100}SI=100P×R×T

Where:

- PPP: Principal amount (initial sum of money)
- RRR: Annual rate of interest (percentage)
- TTT: Time period in years

2. Java Swing Overview

Swing is a part of Java's Standard Library used to create GUI-based applications. It is part of the Java Foundation Classes (JFC) and provides:

- A rich set of components like JLabel, JButton, JTextField, etc.
- Lightweight components that do not depend on the underlying OS.
- Pluggable look and feel, enabling customization of the UI.

3. Components Used in the Application

1. JFrame: The top-level container to create the main application window.
2. JLabel: Displays text to describe input fields.
3. JTextField: Allows users to input data such as principal, rate, and time.
4. JButton: Triggers specific actions such as calculating or clearing inputs.
5. JOptionPane: Displays dialog boxes for error messages or additional information.

4. Event Handling in Swing

- ActionListener Interface:

This interface handles user interactions with components like buttons. For example, when the "Calculate" button is clicked, an ActionEvent is triggered, and the listener executes the associated logic.

5. Input Validation

- Validates that the user inputs numeric values for principal, rate, and time.
- Displays an error message for invalid inputs using JOptionPane.

6. Error Handling

- Ensures robust application behavior by catching NumberFormatException for invalid user inputs.
- Prevents crashes and guides the user to enter valid data.

7. Workflow of the Application

1. The user inputs values for the principal amount, rate of interest, and time period in text fields.
2. Upon clicking the "Calculate" button:
    - Inputs are validated.
    - Simple interest is computed using the formula.
    - The result is displayed in a read-only text field.
2. The "Clear" button resets all fields to allow new inputs.

8. Practical Applications

This application showcases the use of Java Swing for financial calculations and can be extended to include:

- Compound interest calculations.
- Exporting results to files.
- Advanced GUI features like graphs and charts.

Algorithm/Steps:

1. Set Up Environment: Install and configure the JDK.
2. Code Implementation:
    - Create a JFrame with GridLayout.
    - Add labels and text fields for user input.
    - Add buttons for calculation and clearing fields.
    - Implement the ActionListener to handle button clicks.

2. Test the Application:
    - Run the application.
    - Input valid and invalid values to ensure functionality.

Conclusion:

A Java GUI application for calculating simple interest provides an interactive and user-friendly way to compute financial results based on user input.

Frequently Asked Questions:

1. What are layout managers in Java Swing? What problems may occur if layout managers are not used?
2. Describe the role of containers in Java Swing. How do containers help in organizing GUI components?
3. What is meant by Look and Feel in Java Swing? How does it improve the user experience of GUI applications?
4. Discuss the importance of user interface consistency in Swing applications. How does Swing help achieve this consistency?
5. Explain how Swing supports reusability and modularity in GUI design.

Assignment No. 4

Assignment Title: Create a Java servlet that takes input from a web form, calculates the area of a circle based on the provided radius, and displays the result on a web page.

Aim: To create a Java servlet that takes user input for the radius of a circle from a web form, calculates the area, and displays the result on a web page.

Pre-Requisites:

1. Basic understanding of Java and servlets.
2. Familiarity with HTML forms and HTTP request/response mechanism.
3. Java Development Kit (JDK) installed.
4. A servlet container like Apache Tomcat.
5. Integrated Development Environment (IDE) like Eclipse.

Objective:

1. To develop a servlet that accepts user input (radius of the circle).
2. To compute the area of the circle using the formula: Area = π * radius^2.
3. To display the result dynamically on a web page.

Outcomes:

1. A functional Java servlet that performs calculations based on user input.
2. Understanding of how Java servlets interact with HTML forms and the web browser.

Theory:

1. **Client-Side (HTML Form):**
   - An HTML form collects the user input. The form contains a text field for the user to enter the radius of the circle and a submit button to send the data to the server.
   - The user's input is sent to the server through an HTTP request, usually a POST request, which passes the input data as parameters to the servlet.
2. **Server-Side (Java Servlet):**
   - The Java servlet receives the input data from the form through the HTTP request. It then reads the radius value, performs the calculation, and generates the appropriate response.
   - The servlet uses Java's Math.PI constant to represent the value of π and computes the area using the formula mentioned above.

- The servlet then creates an HTTP response, which includes an HTML page displaying the calculated area to the user.

2. **Servlet Lifecycle:**
   - When the user submits the form, the web server invokes the servlet's doPost method (if using POST) or doGet method (if using GET), depending on the form submission method.
   - Inside the method, the servlet reads the form data, processes it, and sends back an HTML response with the calculated result.
2. **Communication Between Client and Server:**
   - The client (browser) sends an HTTP request to the servlet, and the servlet processes the request and generates an HTTP response. The response typically contains dynamic content, such as the area calculation result.
   - This process allows the user to interact with the application in real-time through a web page.

Key Concepts:

- HTTP Request and Response: Servlets handle HTTP requests (from the client) and send HTTP responses back (to the client). The request contains data sent by the user, and the response contains the result (in this case, the area of the circle).
- Form Handling: Web forms are used to collect input from the user. This input is sent as part of the HTTP request when the form is submitted.
- Servlets and HTTP Methods: Servlets use methods like doGet() and doPost() to handle the incoming HTTP request. doPost() is typically used for handling form submissions as it allows data to be sent in the request body (more secure for sensitive data).
- Dynamic Content Generation: The servlet dynamically generates HTML content based on the calculation results, allowing users to view the updated information on the webpage.

Benefits of Using Servlets:

- Platform Independence: Since servlets are written in Java, they can run on any platform that supports Java.
- Scalability: Servlets can handle multiple requests concurrently and can be easily scaled in a web application environment.
- Extensibility: Servlets can be extended to perform various tasks, making them reusable for other calculations or functionalities beyond this simple example.

Algorithm/Steps:

1. Start

   Initialize the servlet environment and set up necessary configurations such as the servlet container (e.g., Apache Tomcat)

2. Create an HTML Form

- Design an HTML form that includes:
  - A text input field to enter the radius of the circle.
  - A submit button to send the form data to the servlet.

0. Submit the Form

- When the user enters the radius and clicks the submit button, the form sends an HTTP request (usually a POST request) to the servlet.
- The radius value entered by the user is sent as a form parameter to the servlet.

0. Create a Java Servlet

- Create a servlet class (e.g., CircleAreaServlet) that will handle the POST request from the form.
- The servlet should extend HttpServlet and override the doPost() method (or doGet() if using GET).

0. Retrieve Form Input from the HTTP Request

- Inside the doPost() method of the servlet, retrieve the value of the radius from the form data.
  - Use request.getParameter("radius") to get the input value.

0. Validate Input

- Check if the radius is a valid, positive number.
- If the input is invalid (e.g., a negative or non-numeric value), display an error message to the user.

0. Calculate the Area of the Circle

- If the input is valid, calculate the area of the circle using the formula: Area=π×(radius)2\text{Area} = \pi \times (\text{radius})^2Area=π×(radius)2
- Use Math.PI for the value of π in the formula.

0. Generate the HTML Response

- After performing the calculation, generate an HTML response that includes the calculated area and display it on the webpage.

- The response should also contain information like the radius entered by the user and the result of the calculation.

0. Send Response to the Client

- Send the HTML response back to the client (browser) to display the result.

Conclusion:

This assignment demonstrates the use of Servlet to build a simple web application for performing mathematical calculations.

Frequently Asked Questions

1. What is a Java Servlet?
2. How do servlets handle user input from web forms?
3. What is required to run a servlet?
4. How do I deploy the servlet?
5. How do I handle invalid input, like text instead of a number?

Assignment No. 5

Assignment Title: Java web application using JSP (Java Server Pages).

Aim: Create a Java web application using JSP (Java Server Pages) that takes input from a user, calculates the factorial of the provided number, and displays the result on a web page.

Pre-Requisites:

1. Understanding of core Java concepts like loops, functions, and data types.
2. Knowledge of Java Server Pages (JSP) and Servlets.
3. Familiarity with setting up and deploying a Java web application using an IDE like Eclipse/NetBeans or a server like Apache Tomcat.
4. Basic knowledge of HTML for creating forms.
5. Understanding of deployment descriptors (web.xml).

Objective:

1. To develop a dynamic web application using JSP to accept user input, process it on the server side, and dynamically display the result on a web page.

Outcomes:

1. Ability to create a web application using JSP.
2. Practical understanding of form handling in JSP.
3. Implementing server-side business logic in a web application.
4. Skills in deploying and testing web applications on a local server.

Theory:

Java Server Pages (JSP):

JSP is a server-side technology used to create dynamic web content. It allows developers to embed Java code directly into HTML, enabling the creation of web applications that respond to user input and dynamically generate content. JSP is an integral part of the Java EE platform and is built on top of Servlets, which makes it more convenient for web development. It provides features like tag libraries, expression language, and script lets, which simplify the development process and improve productivity.

In a typical web application, JSP pages act as the view layer in the MVC (Model-View-Controller)

architecture. The client sends a request through the browser, the server processes the request, and JSP renders the response dynamically based on the server-side logic.

Factorial Calculation:

The factorial of a non-negative integer n is the product of all positive integers less than or equal to n. It is denoted as n! For example:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Factorial has numerous applications in mathematics, computer science, and statistics, including permutations, combinations, and probability calculations. It is defined recursively, where:

$n! = n \times (n-1)!$ AND $0! = 1$

In this application, we calculate the factorial of a number provided by the user using Java logic embedded in a JSP page.

Form Handling in JSP:

Forms in web development are used to collect input from users. In JSP, we use HTML forms to collect data, and the server processes the input through JSP or Servlets. The data from the form is sent to the server using the HTTP methods:

- **GET Method:** The form data is appended to the URL as a query string. It is visible to the user and has length restrictions.
- **POST Method:** The form data is sent in the request body, making it secure and suitable for larger data.

In this assignment, the input number is collected through a form in index.jsp and sent to result.jsp using the POST method. The request.getParameter("parameterName") method is used to retrieve the submitted value.

Lifecycle of a JSP Page:

The lifecycle of a JSP page ensures it is executed efficiently and consistently. It includes the following steps:

1. **Translation Phase:** The JSP file is converted into a Servlet class.
2. **Compilation Phase:** The Servlet class is compiled into a .class file.
3. **Initialization Phase:** The Servlet is initialized and prepared to handle requests.
4. **Execution Phase:** The JSP page processes user requests and dynamically generates responses.
5. **Destruction Phase:** The Servlet is removed from memory when no longer needed.

This lifecycle ensures that JSP pages are compiled and optimized for performance during execution.

Advantages of JSP Over Static HTML:

- **Dynamic Content Generation:** Unlike static HTML, JSP can create web pages that change based on user input or server-side logic.
- **Integration with Java:** JSP allows seamless integration of Java logic with web design, providing powerful functionality.
- **Ease of Maintenance:** Changes can be made directly to JSP pages without recompiling the entire application.
- **Separation of Concerns:** JSP allows developers to separate business logic from presentation using JavaBeans or Servlets.

Backend Processing in JSP:

JSP allows embedding Java code within special tags:

- **Scriptlets (<% %>):** Contain Java code for processing logic.
- **Declarations (<%! %>):** Define methods and variables.
- **Expressions (<%= %>):** Output the result of an expression directly to the client.

For example, in this project, the factorial of a number is calculated using a for loop in a scriptlet block, and the result is displayed using an expression block.

JSP and Web Servers:

JSP pages run on a web server, such as Apache Tomcat, which interprets the JSP code, executes the embedded Java logic, and serves the generated HTML content to the client's browser. The server acts as a bridge between the client (browser) and the application logic. By understanding these fundamental concepts, you can create a dynamic web application like the factorial calculator with JSP.

Algorithm/Steps:

Setup the Environment:

- Install Apache Tomcat and set up a dynamic web project in an IDE like Eclipse.

Design the Web Form:

- Create an HTML form in index.jsp to accept a number input from the user.

Capture User Input:

- Use the request.getParameter() method in JSP to retrieve the input.

Process the Input:

- Write a JSP scriptlet to calculate the factorial of the given number.

Display the Output:

- Use JSP expressions to display the result dynamically on the web page.

Deploy and Test the Application:

- Deploy the application on Apache Tomcat and test it by providing input through the browser.

Conclusion:

This assignment demonstrates the use of JSP to build a simple web application for performing mathematical calculations. It enhances the understanding of form handling, server-side logic processing, and dynamic content generation.

Frequently Asked Questions:

1. What is JSP used for in a web application?
2. Why is JSP better than static HTML?
3. How does JSP handle user input?
4. What is JSP Expression Language?
5. What are JSP custom tags?

Assignment No. 6

Assignment Title: Implementing a Simple Model-View-Controller (MVC) Architecture for an Online Bookstore

Aim: Implement a simple Model-View-Controller (MVC) architecture for a Java application that simulates a basic online bookstore. The application should allow users to view a list of available books, add books to their cart, and view the contents of their cart.

Pre-Requisites:

1. Familiarity with the MVC architecture.
2. Knowledge of Java Collections Framework (e.g., ArrayList).

Objective:

1. To demonstrate the separation of concerns in software design by implementing the MVC architecture.
2. To create a functional application that allows users to interact with a simulated online bookstore.
3. To apply Java concepts like collections, encapsulation, and event handling effectively.

Outcomes:

1. Successfully implement a Java application with clear separation of the Model, View, and Controller.
2. Gain practical experience in building structured and maintainable code.
3. Develop skills to handle user inputs and manipulate data dynamically.

Theory:

The Model-View-Controller (MVC) architecture is a design pattern used for developing user interfaces. It separates the application into three interconnected components:

1. **Model**: Represents the application's data and business logic. It manages the state of the application and communicates directly with the Controller.
2. **View**: Displays the data (from the Model) and sends user interaction inputs to the Controller.
3. **Controller**: Acts as an intermediary between the Model and the View. It processes user inputs, updates the Model, and refreshes the View.

For this application, the **Model** will store the list of books and the cart, the **View** will handle displaying data and receiving input, and the **Controller** will manage the application's flow.

Algorithm/Steps:

- **Model**:

- Create classes for Book and Cart.
- Define attributes like title, author, and price for Book.
- Implement methods to add, retrieve, and manipulate books and cart data.

- **View**:
  - Create methods to display the list of books and the cart's contents.
  - Provide methods to capture user input (e.g., book selection or action commands).
- **Controller**:
  - Implement logic to handle user inputs and perform actions like adding a book to the cart or displaying the cart's contents.
  - Ensure proper communication between the Model and View.
- **Main Application**:
  - Initialize the Model, View, and Controller.
  - Start the application loop to interact with the user.

Conclusion:

The application demonstrates the effective use of the MVC architecture in Java. By separating concerns, the code is modular, maintainable, and easy to extend for future functionalities like payment systems or user authentication.

Frequently Asked Questions:

1. What is the purpose of the MVC architecture?
2. Why use an ArrayList for storing books?
3. If this application were to be deployed as a web-based system, explain how the MVC components would change or adapt. What additional technologies or frameworks would be needed?
4. Analyze the advantages and limitations of using the MVC architecture in this project. Under what circumstances might MVC not be the best choice for a small-scale application like this one?

Assignment No. 7

Assignment Title

Creating a Basic Java Project Using Apache Maven with Standard Directory Structure

Aim

To create a basic Java project using **Apache Maven**, following the **standard Maven directory structure**, and to configure the project using a pom.xml file with at least one external dependency.

Pre-Requisites

1. Basic understanding of Java programming
2. Familiarity with build tools and project structure
3. Knowledge of XML basics
4. Awareness of dependency management concepts

Objectives

1. To understand the role of **Apache Maven** in Java project management
2. To create a Java project following the **standard Maven directory structure**
3. To learn how to configure project metadata using pom.xml
4. To add and manage external libraries using Maven dependencies

Outcomes

1. Successfully create a Java project using Apache Maven
2. Understand the standard Maven project layout
3. Gain hands-on experience with dependency management using pom.xml
4. Develop structured and maintainable Java applications

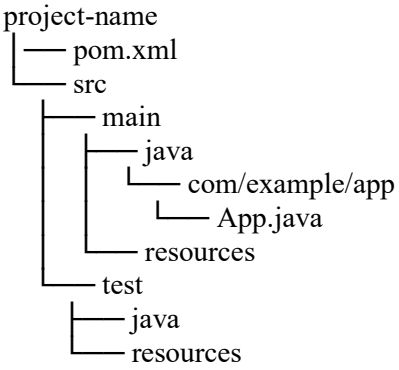
Theory

Apache Maven is a **build automation and project management tool** primarily used for Java projects. It simplifies the build process by using a **Project Object Model (POM)**, which is an XML file (pom.xml) that contains project configuration details.

Maven provides:

- A **standard directory structure**
- **Automatic dependency management**
- Easy project build and lifecycle management

Standard Maven Directory Structure



This structure ensures consistency, readability, and easier collaboration among developers.

Algorithm / Steps

Step 1: Create a Maven Project

- Use Maven command or IDE option to create a new Maven project.
- Provide Group ID, Artifact ID, and Version.

Step 2: Follow Standard Maven Directory Structure

- Place application source code inside src/main/java
- Place test code inside src/test/java
- Store configuration files in resources folders

Step 3: Configure pom.xml

- Define project metadata such as:
  - Group ID
  - Artifact ID
  - Version
- Specify Java compiler version

Step 4: Add Dependency

- Add at least one external dependency in pom.xml
  Example: Apache Commons or Log4j for utility or logging support.

Step 5: Implement a Basic Java Class

- Create a simple Java class to verify project setup
- Run the project using Maven commands

Sample Dependency Configuration (Conceptual)

- Dependency is added inside the <dependencies> tag in pom.xml
- Maven automatically downloads required libraries from the central repository

Conclusion

This assignment demonstrates how Apache Maven simplifies Java project development by enforcing a standard structure and managing dependencies efficiently. By using Maven, developers can focus more on application logic rather than manual configuration, resulting in clean, maintainable, and scalable projects.

Frequently Asked Questions (FAQs)

1. What is the purpose of Apache Maven?

2. Why is a standard Maven directory structure important?

3. What is the role of the pom.xml file?

4. How does Maven handle dependencies automatically?

5. What are the advantages and limitations of Maven?

Assignment No. 8

Assignment Title: Create a simple web application using the Spring Framework that allows users to manage a list of tasks.

Aim: Create a simple web application using the Spring Framework that allows users to manage a list of tasks. The application should provide functionality to view existing tasks, add new tasks, mark tasks as completed, and delete tasks.

Pre-Requisites:

1. **Java Development Kit (JDK)**: Version 11 or higher.
2. **Maven** (Optional if you use an IDE like IntelliJ)
3. **IDE**: Use IntelliJ IDEA, Eclipse, or Visual Studio Code for easier development.
4. **Spring Boot CLI** (Optional): Install it if you want an alternative to Maven for running the application.

Objective:

1. Understand Spring framework
2. Create simple web application using Spring Framework

Outcomes:

1. Students able to understand Spring framework
2. Students able to create simple web application using Spring Framework

Theory:

Spring Framework is a comprehensive and versatile platform for enterprise Java development. It is known for its Inversion of Control (IoC) and Dependency Injection (DI) capabilities that simplify creating modular and testable applications. Key features include Spring MVC for web development, Spring Boot for rapid application setup, and Spring Security for robust authentication and authorization. With a rich ecosystem covering Spring Data for database interactions and Spring Cloud for building microservices, Spring supports scalable and resilient enterprise solutions, making it an essential framework for developers of all experience levels.

What is Spring Framework?

Spring is a lightweight and popular open-source Java-based framework developed by Rod Johnson in 2003. It is used to develop enterprise-level applications. It provides support to many other frameworks such as Hibernate, Tapestry, EJB, JSF, Struts, etc. so it is also called a framework of frameworks. It's an application framework and IOC (Inversion of Control) container for the Java platform. The spring contains several modules like IOC, AOP, DAO, Context, WEB MVC, etc.

Why to use Spring?

Spring framework is a Java platform that is open source. When it comes to size and transparency, Spring is a featherweight. Spring framework's basic version is about 2MB in size. The Spring Framework's core capabilities can be used to create any Java program, however there are modifications for constructing web applications on top of the Java EE platform. By offering a POJO-based programming model, the Spring framework aims to make J2EE development easier to use and to promote good programming habits.

Algorithm/Steps:

Algorithm for the Task Management Application

1. Initialize the Application:
    - Set up a Spring Boot application.
    - Include dependencies for Spring Web, Spring Data JPA, Thymeleaf, and an embedded H2 database.
2. Define the Task Model:
    - Create a Task class to represent a task entity with fields like id, name, status, and description.
2. Set Up the Database:
    - Use H2 as an in-memory database for storing tasks.
    - Configure the database in application.properties.
2. Create the Repository Layer:
    - Define a TaskRepository interface that extends JpaRepository for basic CRUD operations.
2. Create the Service Layer:
    - Implement a TaskService class to handle business logic, such as retrieving tasks, saving tasks, marking tasks as completed, and deleting tasks.
2. Create the Controller Layer:
    - Define a TaskController to handle HTTP requests for viewing tasks, adding tasks, marking them as completed, and deleting them.
2. Create the Frontend:
    - Use Thymeleaf templates for the user interface to display tasks and handle form submissions.
2. Run and Test:

- Start the application.
- Use a browser to access the UI and test all functionalities.

Conclusion :

The web application developed using the Spring Framework provides an efficient and modern solution for task management. It showcases how Spring simplifies web application development by integrating key modules such as Spring MVC for handling HTTP requests, Spring Boot for rapid application setup, and Spring Data JPA for interacting with a database**.**

Frequently Asked Questions

1. What is the Spring Framework?
2. What is Spring Boot, and why is it used?
3. What is Spring MVC?
4. What is the benefit of using RESTful APIs in this application?
5. How can I deploy the Spring application to production?

Assignment No. 9

Assignment Title

Create a Java Application Using Hibernate Framework

Aim

Create a Java application using Hibernate framework to manage a simple database of students. The application should allow users to perform CRUD operations (Create, Read, Update, Delete) on the student records stored in the database.

Pre-Requisites

1. Java Development Kit (JDK): Version 8 or higher
2. Basic knowledge of Java programming
3. Understanding of Object-Oriented Programming concepts
4. Basic knowledge of relational databases and SQL
5. IDE: Eclipse
6. MySQL / H2 / Oracle database (any one)

Objectives

1. To understand the fundamentals of the **Hibernate ORM framework**
2. To implement database operations using Hibernate

Outcomes

1. Students will be able to understand Hibernate architecture and workflow
2. Students will gain hands-on experience in performing CRUD operations

Theory

Hibernate is a **powerful Object-Relational Mapping (ORM) framework** for Java that simplifies interaction with relational databases. Instead of writing complex SQL queries, developers can interact with the database using **Java objects**, allowing cleaner and more maintainable code.

Hibernate manages:

- Database connections
- SQL generation
- Object mapping
- Transaction management

What is Hibernate Framework?

Hibernate is an open-source Java framework that provides a solution to map **Java objects (POJOs)** to **database tables** and vice versa. It is part of the Java Persistence ecosystem and acts as a bridge between object-oriented programming and relational databases.

Why Use Hibernate?

- Reduces boilerplate JDBC code
- Database-independent
- Supports automatic table generation
- Improves performance using caching
- Simplifies CRUD operations

Algorithm / Steps

Algorithm for Student Management Application

1. Initialize the Application

- Create a Java project.
- Add Hibernate libraries and database driver dependencies.
- Configure Hibernate using hibernate.cfg.xml.

2. Define the Student Model

- Create a Student class.
- Define attributes such as:
    - Student ID
    - Name
    - Department
    - Email
- Annotate the class using Hibernate annotations like @Entity, @Id, and @Column.

3. Configure Database Connectivity

- Specify database URL, username, password, and dialect in Hibernate configuration.
- Enable automatic table creation if required.

4. Create Hibernate Utility Class

- Configure SessionFactory.
- Manage Hibernate sessions efficiently.

5. Implement CRUD Operations

- **Create:** Insert new student records into the database.
- **Read:** Retrieve student details using ID or fetch all students.
- **Update:** Modify existing student records.
- **Delete:** Remove student records from the database.

6. Transaction Management

- Begin a transaction before performing operations.
- Commit the transaction after successful execution.
- Rollback in case of failure.

7. Execute and Test

- Run the application.
- Verify that all CRUD operations work correctly on the database.

Conclusion

The Student Management application developed using the Hibernate framework demonstrates the effective use of ORM for database operations in Java. Hibernate simplifies data persistence by eliminating repetitive JDBC code and enabling developers to work directly with Java objects. The application is modular, maintainable, and can be easily extended to include advanced features such as validation, pagination, or integration with Spring.

Frequently Asked Questions (FAQs)

1. What is Hibernate Framework?

2. What is ORM, and why is it important?

3. How does Hibernate differ from JDBC?

4. What is a Session in Hibernate?

5. How can this application be extended further?

Assignment No. 10

Assignment Title

Create a REST API Using Spring Boot

Aim

Create a REST API to add employees to the employee list and get the list of employees. In order to do this, first create a simple Spring Boot project in any of the IDE's

Pre-Requisites

1. Java Development Kit (JDK): Version 8 or higher
2. Basic knowledge of Java programming
3. Understanding of Object-Oriented Programming concepts
4. Basic understanding of RESTful services
5. IDE: Eclipse
6. Maven (optional if IDE supports it internally)

Objectives

1. To understand the basics of **Spring Boot framework**

Outcomes

1. Students will be able to understand the architecture of Spring Boot applications

Theory

Spring Boot is a **lightweight framework built on top of the Spring Framework** that simplifies the development of standalone, production-ready applications. It eliminates boilerplate configuration and provides embedded servers such as Tomcat, making application deployment easier.

REST (Representational State Transfer) is an architectural style used for designing networked applications. RESTful services use standard HTTP methods like **GET, POST, PUT, and DELETE** to perform operations on resources.

What is Spring Boot?

Spring Boot is an open-source Java-based framework that allows developers to create **standalone Spring applications** with minimal configuration. It follows the principle of **convention over configuration** and provides built-in support for REST APIs.

Why Use Spring Boot for REST APIs?

- Simplifies application setup
- Embedded web server (no external deployment needed)
- Easy REST API development using annotations
- Faster development and testing
- Production-ready features

Algorithm / Steps
Algorithm for Employee Management REST API

1. Initialize the Application

- Create a Spring Boot project using IDE or Spring Initializr.
- Add dependencies such as:
    - Spring Web
- Configure application properties if required.

2. Define the Employee Model

- Create an Employee class.
- Define attributes such as:
    - Employee ID
    - Name
    - Department
    - Salary
- Use appropriate annotations to define the model.

3. Create Employee Repository / Storage

- Use an in-memory list (such as ArrayList) to store employee data.
- Implement methods to add and retrieve employee records.

4. Create REST Controller

- Create a controller class annotated with @RestController.
- Define REST endpoints:
    - POST request to add a new employee
    - GET request to retrieve all employees

5. Handle HTTP Requests

- Use @PostMapping to accept employee data.
- Use @GetMapping to return the employee list.
- Exchange data in JSON format.

6. Run and Test the Application

- Run the Spring Boot application.
- Test the APIs using:
    - Browser
    - Postman
    - cURL

Conclusion

The Employee Management REST API developed using Spring Boot demonstrates how RESTful web services can be created efficiently with minimal configuration. Spring Boot simplifies backend development by providing embedded servers and annotation-based programming. This application can be easily extended to support full CRUD operations, database integration, and security features.

Frequently Asked Questions (FAQs)

1. What is Spring Boot?

2. What is a REST API?

3. What HTTP methods are used in this application?

4. How does Spring Boot simplify REST API development?

5. How can this application be extended further?